

ОТЧЁТ ПО ЭТАПУ 8: ИНСПЕКТИРОВАНИЕ И МОДИФИКАЦИЯ МОДУЛЕЙ ПРОЕКТА

Краткое описание проекта и перечень проверяемых модулей

Анонимный видеохостинг для учебных целей. Технологии: C# / Avalonia (клиент), ASP.NET Core (сервер), PostgreSQL (БД). Разработка ведётся в VS Code с расширениями C#, C# Dev Kit, Templates for Avalonia, XAML Styler.

Проверке подвергнуты следующие модули программной системы:

- Модуль авторизации (клиентская и серверная части);
- Модуль видео (загрузка и потоковое воспроизведение);
- Модуль комментариев;
- Модуль лайков.
- Серверные контроллеры (VideosController, CommentsController, LikesController, PlaylistsController, ModerationController)
- Модели и DTO (VideoDto, CommentDto, LikeDto)
- Сервисы (VideoService, AuthService)

Выбор модулей обусловлен их критической важностью для осуществления основного функционала системы.

Выбранные стандарты кодирования и правила оформления кода

1. Стандарт оформления C#: официальные рекомендации Microsoft (PascalCase для классов/методов, camelCase для переменных, фигурные скобки на новой строке).
2. Форматирование XAML (Avalonia): используется расширение XAML Styler для автоматического выравнивания атрибутов, отступов и порядка элементов.
3. Статический анализ: встроенный анализатор C# Dev Kit (подсветка ошибок, предупреждений, рекомендаций).
4. Документация: XML-комментарии для публичных методов.

Обнаруженные несоответствия

В результате инспектирования были выявлены следующие нарушения.

Модуль	Проверяемое правило	Выявленное несоответствие
Модуль авторизации	Именованное (camelCase)	Приватное поле <code>_Login</code> вместо <code>_login</code> ; метод <code>GetUserData</code> без

		комментария.
Модуль видео	Обработка ошибок	Отсутствует проверка на пустой file path при загрузке видео.
Модуль комментариев	Длина метода	Метод AddComment превышал 50 строк, смешивал логику валидации, сохранения и уведомления.
Модуль лайков	Форматирование XAML	В файле окна оценки были неравномерные отступы и хаотичный порядок атрибутов.
VideosController	Обработка ошибок	Отсутствовал отдельный метод для инкремента просмотров, что приводило к двойному счёту при открытии в MPV
VideoService	Асинхронность	Метод IncrementViewsAsync не обрабатывал случай, когда видео не найдено (ошибка EF)
Клиент PlayerWindow	Управление внешним процессом	Повторное нажатие "Открыть в MPV" создавало несколько окон
Клиент PlayerWindow	Логика событий	При закрытии MPV происходило лишнее обновление списка видео

Выполненная модификация модулей

- Модуль авторизации: поле переименовано в `_login`, добавлен XML-комментарий к методу `GetUserData`.
- Модуль видео: добавлена проверка: `if (string.IsNullOrEmpty(filePath)) throw new ArgumentException("Путь к файлу не указан");`
- Модуль комментариев: метод `AddComment` разбит на три: `ValidateComment`, `SaveComment`, `NotifyAuthor`.
- Модуль лайков: применён XAML Styler (Alt+Shift+F) для автоматического форматирования XAML-разметки — устранены отступы, упорядочены атрибуты.
- Добавлен endpoint `POST /api/videos/{id}/view` для атомарного учёта просмотров.
- В `VideoService.IncrementViewsAsync` добавлен try-catch и проверка существования видео.
- В `PlayerWindow` добавлен флаг `_videoDeleted` и проверка `IsMpvRunning`.
- Обновлено логика вызова обновления списка только при реальном удалении/закрытии.

Повторная проверка после исправления

После внесения исправлений была проведена повторная проверка:

- Все модули проходят проверку C# Dev Kit без предупреждений.
- XAML Styler не выявил нарушений после автоформатирования.
- XML-комментарии добавлены для всех публичных методов.

Все нарушения из таблицы были устранены.

Вывод

Код модулей приведён к единому стандарту. Использование расширений VS Code (C# Dev Kit, XAML Styler) позволило автоматизировать часть проверок. Модули готовы к дальнейшей разработке и тестированию.